System Design and Software Architecture

AI Code Security Analyzer

Submitted for the Course

CSE4104 - Software Engineering

Department of Computer Science and Engineering
Northern University of Business and Technology, Khulna

1 Team Information

Project Title: AI Code Security Analyzer

Team Name: CSE4104-7C-T04

Team Leader: Sadia Islam Jhumur

1.1 Team Members

No.	Member Name	Student ID
1	Sadia Islam Jhumur (Team Leader)	11230121171
2	Sakil Hasan Saikat	11230121188
3	Tanjilanta Kanchi	11220121034
4	Farin Islam Mim	11230121061

2 Project Information

2.1 Project Overview

AI Code Security Analyzer is an AI-powered Software-as-a-Service platform developed to identify security vulnerabilities in source code before deployment. The system enables users to upload source code files, paste source code directly, or submit GitHub repository URLs for automated analysis.

2.2 Problem Statement

Software vulnerabilities remain one of the primary causes of cyberattacks, data breaches, and unauthorized access. Existing static analysis tools often rely on predefined rules and signatures, limiting their ability to identify contextual and logic-based vulnerabilities.

2.3 Objectives

1. Automate source code security analysis.
2. Detect common and advanced vulnerabilities.
3. Generate remediation recommendations.
4. Support GitHub repository analysis.
5. Improve secure software development practices.

2.4 Scope

The platform supports source code uploads, repository scanning, AI-powered vulnerability analysis, report generation, and scan history management.

3 System Architecture

The proposed architecture follows a multi-layered design consisting of a frontend interface, backend services, database infrastructure, and AI security analysis components. This separation improves scalability, maintainability, and reliability.

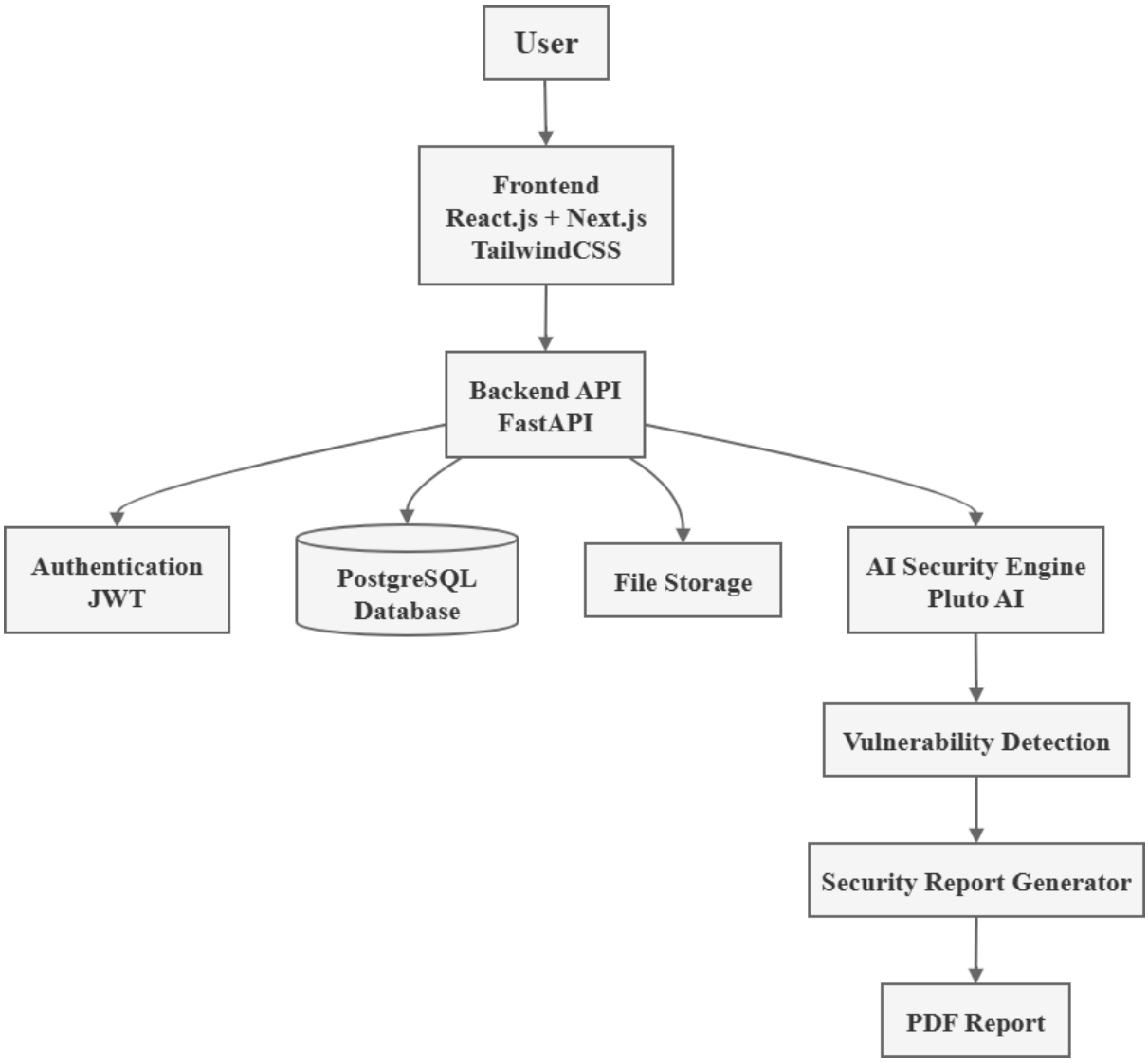


Figure 1: System Architecture of the AI Code Security Analyzer Platform

The architecture enables secure communication between users, backend services, databases, and AI providers. Its modular structure supports future scalability, maintainability, and integration of additional AI services.

4 Database Design

The database has been designed to support user management, project organization, source code storage, vulnerability tracking, report generation, and system activity monitoring.

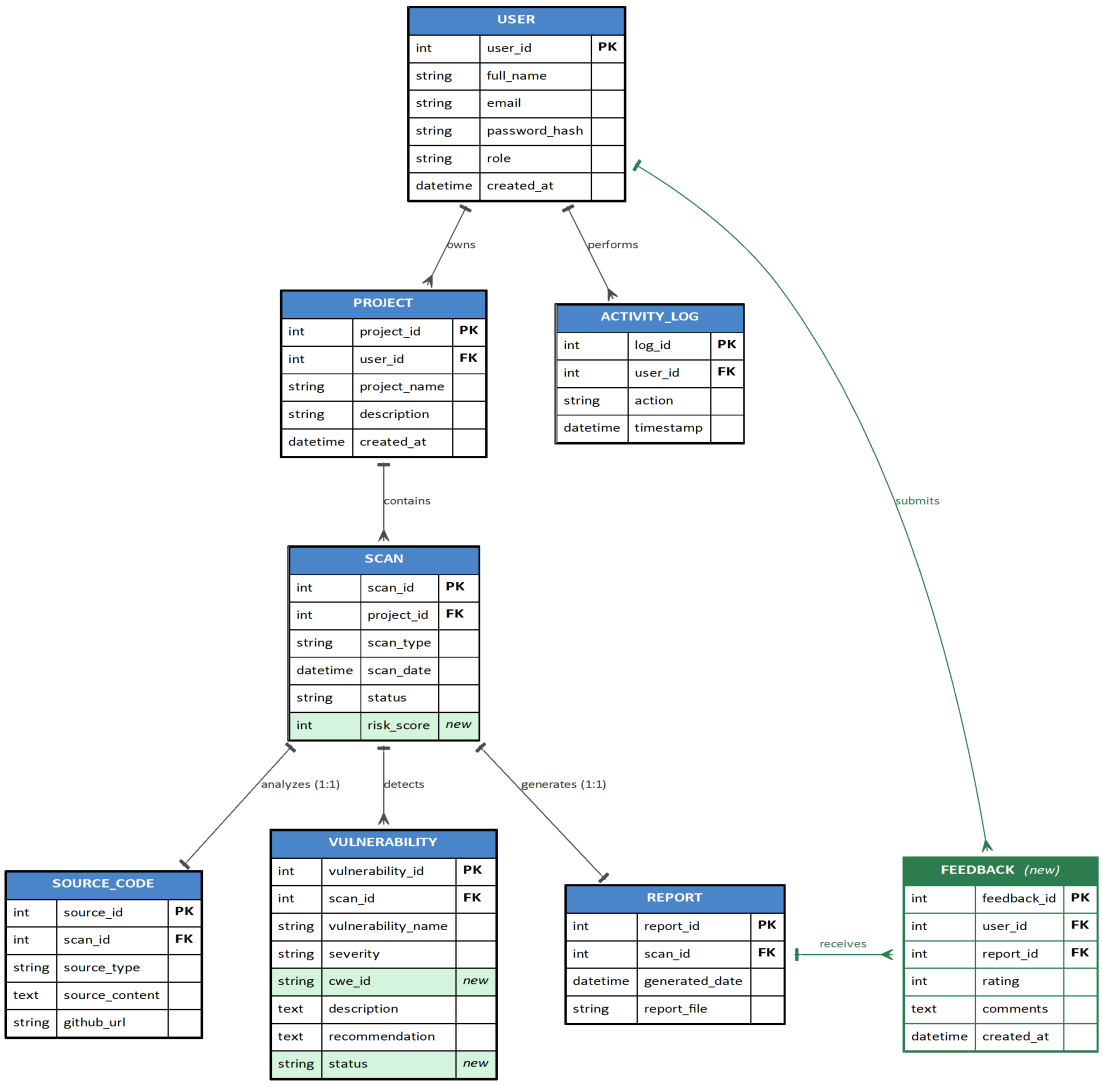


Figure 2: Updated Entity Relationship Diagram of the AI Code Security Analyzer System

The database schema ensures efficient storage and retrieval of project data, scan results, vulnerability findings, and generated reports. The relationships between entities support all functional requirements identified in the system analysis phase.

5 Use Case Analysis

The use case diagram illustrates interactions between users and the AI Code Security Analyzer platform.

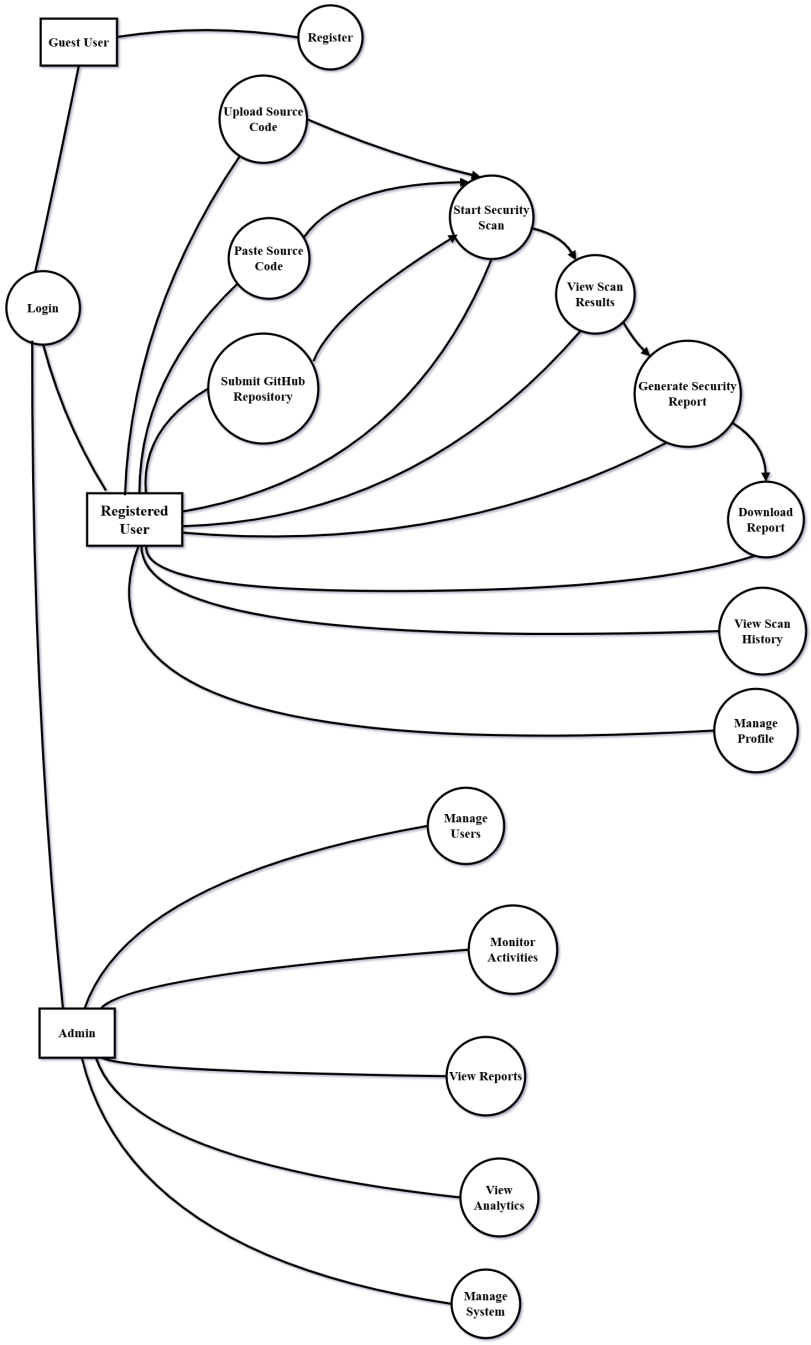


Figure 3: Use Case Diagram of AI Code Security Analyzer

The use case model illustrates the interactions between system actors and core functionalities. It provides a clear representation of user activities, administrative responsibilities, and AI-assisted operations.

6 Activity Diagram

The activity diagram presents the workflow of AI-powered source code security analysis from user submission to report generation.

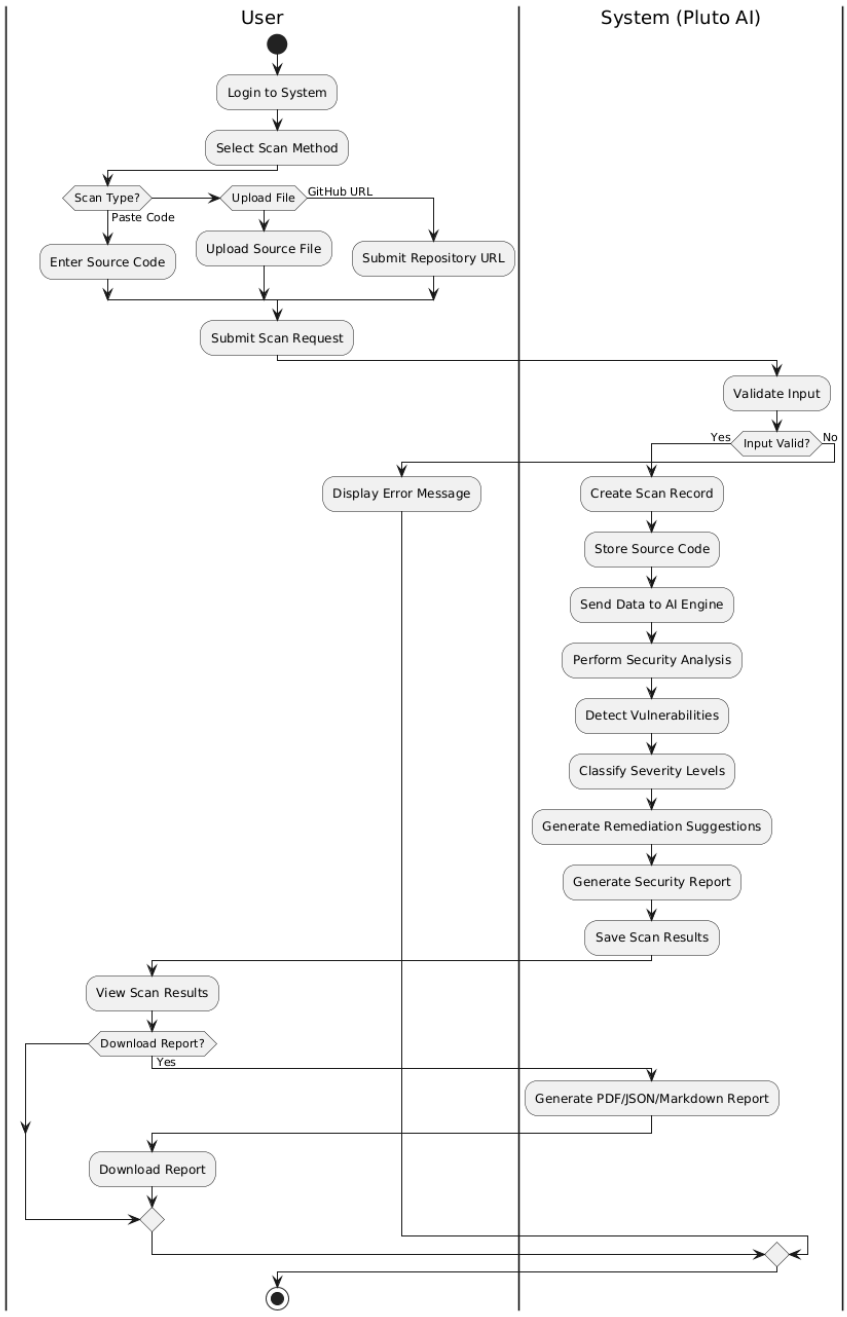


Figure 4: Swimlane Activity Diagram for AI-Powered Source Code Security Analysis Workflow

The activity workflow demonstrates how source code is processed from submission to report generation. It highlights the collaboration between the user, backend services, and AI Security Engine during vulnerability analysis.

7 API Design

The AI Code Security Analyzer follows a RESTful API architecture that enables secure communication between the frontend application, backend services, database infrastructure, and AI Security Engine.

7.1 Authentication APIs

Method	Endpoint	Purpose
POST	/api/v1/auth/register	Create a new user account in the system
POST	/api/v1/auth/login	Authenticate user credentials and generate access tokens
POST	/api/v1/auth/refresh-token	Generate a new access token using a refresh token
POST	/api/v1/auth/logout	Invalidate the current user session

7.2 Project and Scan APIs

Method	Endpoint	Purpose
POST	/api/v1/projects	Create a project
GET	/api/v1/projects	Retrieve project list
POST	/api/v1/projects/{id}/scans	Submit source code for scanning
GET	/api/v1/scans/{id}	Retrieve scan information
POST	/api/v1/scans/{id}/rescan	Execute a new scan

7.3 AI Analysis APIs

Method	Endpoint	Purpose
POST	/api/v1/scans/{id}/analyze	Trigger AI vulnerability analysis
GET	/api/v1/scans/{id}/vulnerabilities	Retrieve identified vulnerabilities
POST	/api/v1/vulnerabilities/{id}/fix	Generate AI remediation suggestions
POST	/api/v1/ai/chat	Interact with the AI assistant

8 AI Integration Workflow

8.1 Need for Artificial Intelligence

Traditional vulnerability scanners rely on predefined signatures and pattern matching techniques. Although effective for detecting known vulnerabilities, such systems often fail to identify context-dependent security flaws. Artificial Intelligence enables semantic understanding of source code and improves vulnerability detection accuracy.

8.2 AI Models

The proposed system integrates multiple Large Language Models:

- GPT-4o
- Claude
- Gemini
- DeepSeek

8.3 AI Workflow Diagram

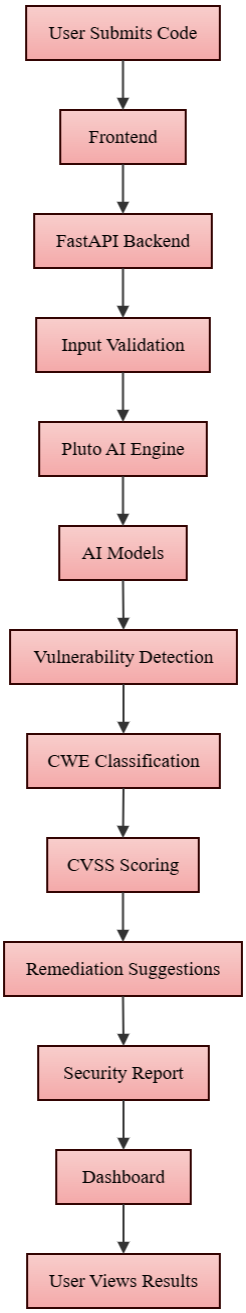


Figure 5: AI Integration Workflow of the AI Code Security Analyzer

8.4 Workflow Description

The workflow begins when a user submits source code through file upload, direct code input, or GitHub repository scanning. The frontend forwards the request to the FastAPI backend, where the source code is validated and preprocessed. The processed data is then sent to the Pluto AI Security Engine, which communicates with GPT-4o, Claude, Gemini, and DeepSeek to perform semantic code analysis.

The AI engine identifies vulnerabilities, maps findings to Common Weakness Enumeration (CWE) categories, calculates severity scores using the Common Vulnerability Scoring System (CVSS), and generates remediation recommendations. Finally, a structured security report is generated and displayed to the user through the frontend dashboard.

8.5 Benefits

- Improved vulnerability detection accuracy.
- Reduced manual security review effort.
- Context-aware security analysis.
- Automated remediation guidance.
- Faster software security assessment.

9 GitHub Repository

The project repository is organized to support collaborative development, version control, documentation management, and future deployment activities.

9.1 Repository Structure

```
frontend/
backend/
database/
documentation/
diagrams/
README.md
```

9.2 Repository Link

`https://github.com/0xSaikat/pluto-ai`

10 References

1. OpenAI API Documentation.
2. Anthropic Claude Documentation.
3. Google Gemini Documentation.
4. DeepSeek API Documentation.
5. OWASP Top 10 Security Risks.
6. CWE Database.
7. CVSS Documentation.
8. FastAPI Documentation.
9. PostgreSQL Documentation.
10. React Documentation.
11. GitHub Documentation.